

Power PMAC 개발 도구 사용자 매뉴얼

Claude Code Skill + MCP — 처음 쓰는 분을 위한 안내서

이 문서 하나로 **설치** → **등록** → **실제 사용**까지 따라 할 수 있습니다.

목차

0. 이 문서는
1. Claude Code란?
2. 이 도구가 주는 것 — Skill과 MCP가 할 수 있는 일
3. 사전 준비
4. 설치·등록
5. 설치 확인
6. 권한 승인 이해하기
7. Skill 사용법 — 물어보기만 하면 됩니다
8. MCP 사용법 — 개발 워크플로
9. 자주 묻는 질문 · 문제 해결
10. 더 보기

0. 이 문서는

- **대상:** Claude Code를 처음 쓰는 Power PMAC 개발자.
- **끝나면 할 수 있는 것**
 - Claude에게 한국어로 Power PMAC 문법·데이터구조·함정을 물어보고 정확한 답과 코드를 받습니다. → **Skill**
 - Claude에게 "프로젝트 빌드해줘", "192.168.0.200에 다운로드해줘", "Motor[1].ActPos 읽어줘"처럼 시켜서 **실제 컨트롤러를 제어**합니다. → **MCP**
- **소요 시간:** 설치 5~10분(+최초 빌드 1회). 이후엔 자연어 대화만.

1. Claude Code란?

Claude Code는 **터미널·IDE에서 동작하는 AI 코딩 에이전트**입니다. 채팅하듯 한국어로 지시하면 파일을 읽고·쓰고, 명령을 실행하고, (이 도구를 설치하면) Power PMAC 컨트롤러까지 다룹니다.

- **설치/실행:** 사내 표준 설치본 또는 공식 안내(claude.com/claude-code)를 따릅니다. 설치 후 터미널에서 `claude` 를 입력하거나 VS Code/IDE 확장으로 실행합니다.
- **핵심 사용법:** **하고 싶은 일을 말로 시키면 됩니다.** Claude가 필요한 도구를 알아서 고르고, 실행 전에 승인을 묻습니다(→ 6장).

이 매뉴얼이 추가로 설치하는 것은 두 가지입니다 — **Skill**(지식)과 **MCP**(컨트롤러 제어).

2. 이 도구가 주는 것 — Skill과 MCP가 할 수 있는 일

Claude Code에 두 가지가 더해집니다 — 지식을 주는 **Skill**과, 실제 장비를 다루는 **MCP**.

Skill `powerpmac-dev` — Power PMAC를 "이해"하게 합니다

질문하면 자동으로 적용됩니다(별도 호출 없음). 할 수 있는 일:

- **정확한 코드 작성** — Script 모션 프로그램(`prog`), Script PLC(`plc`), C(CPLC 실시간 / capp 백그라운드)를 문법에 맞게 생성합니다.
- **코드 리뷰·디버깅** — "왜 안 되는지"를 짚고, 흔한 함정(인덱스는 상수만, 태스크 우선순위, `save` / `reset` , 단위 혼동)을 진단합니다.
- **데이터구조·요소 조회** — `Sys.` / `Motor[]` / `Coord[]` / `Gate3[]` 등 **1,800개 이상 요소**의 의미·SAVED 여부·레거시 I-변수 대응을 설명합니다.
- **개념 설명** — 좌표계(`&`) vs 모터(`#`), 우선순위 4계층(Phase/Servo/RTI/Background), 록어헤드, G코드, CfromScript 등.
- **프로젝트 구조 안내** — `.ppproj` 매니페스트, `pp_proj.ini` 로드 순서, 컨트롤러 `/var/ftp/usrflash/Project` 매핑.
- **검증된 예제 제공** — 매뉴얼에서 추출·검증한 모션·PLC·C·기구학 스니펫을 바로 적용해 줍니다.

근거는 공식 매뉴얼 4종(약 3,260쪽)과 firmware intellisense 요소표에서 정제했습니다. 미심쩍으면 "근거 reference가 어디야?"라고 되물어 출처를 확인할 수 있습니다.

🔗 MCP powerpmac — PC에서 컨트롤러를 "조작"합니다

"~해줘"라고 시키면 Claude가 알맞은 도구를 호출합니다. 할 수 있는 일:

- **빌드** — PDK로 프로젝트를 로컬 컴파일(Script + **ARM C 크로스컴파일**)하고 에러·경고를 보고합니다.
- **다운로드** — 빌드 결과를 컨트롤러로 전송(rsync)한 뒤 로드(**projpp**)합니다. 헤드리스로 약 8초.
- **라이브 조회** — gpascii 세션으로 **Motor[1].ActPos** 같은 값을 실시간으로 읽습니다. 여러 개를 한 번에(batch)도 가능.
- **명령 실행** — 대입·조그·프로그램 실행 등 온라인 명령을 보냅니다(**Motor[1].JogSpeed=10** , **#1j+** , **save** ...).
- **셸 접근** — 컨트롤러의 리눅스 명령(파일 목록, 로그 확인 등)을 실행합니다.
- **전체 루프** — 한 대화 안에서 **작성 → 빌드 → 다운로드 → 검증 → 저장**까지 이어집니다(→ 8장 플로차트).

조건

- **Skill만** 쓰려면(코드 작성·리뷰) PDK·SDK 없이도 됩니다 → 4장 **-SkillOnly**.
- MCP는 **Power PMAC IDE + PDK가 설치된 Windows PC**가 필수입니다(빌드·다운로드에 컴파일러·라이선스가 필요).

3. 사전 준비

항목	용도	없으면
Claude Code	모든 기능의 실행 환경	먼저 설치(1장)
Windows + .NET Framework 4.8	MCP 실행(Win10/11 기본 포함)	—
Power PMAC IDE + PDK 라이선스	MCP 빌드·다운로드 (컴파일러·rsync·라이선스 제공)	MCP 불가 → Skill만(-SkillOnly)
.NET SDK (최초 1회)	MCP 빌드	MCP 빌드 불가 → Skill만

코드 작성·리뷰만 하는 분은 IDE/PDK·SDK 없이 **Skill만** 쓰면 됩니다.

4. 설치·등록

⚠ **MCP를 쓰려면 Power PMAC IDE + PDK 설치가 필수입니다.** PDK가 제공하는 컴파일러·rsync·라이선스(`CLLLicFile.lic`)가 있어야 빌드·다운로드가 됩니다. PDK가 없는 PC는 아래 **Skill만 설치**(`-SkillOnly`)로 코드 작성·리뷰만 사용하세요.

사내 저장소를 clone하고 `setup.ps1` 을 한 번 실행하면 **Skill 설치 · PDK/컴파일러 자동 감지 · MCP 빌드 · 등록**까지 자동으로 처리됩니다. 경로는 자동 감지되어 직접 편집할 필요가 없습니다.

Skill + MCP (PDK 설치된 PC):

```
git clone https://github.com/minimin333/PowerPMAC_MCP.git C:\Tools\PowerPMAC_MCP
cd C:\Tools\PowerPMAC_MCP
powershell -ExecutionPolicy Bypass -File .\setup.ps1
```

Skill만 (PDK 없는 PC):

```
git clone https://github.com/minimin333/PowerPMAC_MCP.git C:\Tools\PowerPMAC_MCP
cd C:\Tools\PowerPMAC_MCP
powershell -ExecutionPolicy Bypass -File .\setup.ps1 -SkillOnly
```

끝나면 **Claude Code**를 재시작하세요. (상세 절차·문제 해결은 루트 `INSTALL.md` 참고.)

5. 설치 확인

- 터미널에서 `claude mcp list` → 목록에 `powerpmac` 이 보이면 MCP 등록 완료.
- Claude Code를 재시작하고 다음을 물어보세요(= Skill 확인):

"Power PMAC에서 Motor[1] 데이터 구조 간단히 설명해줘"

실제 요소(예: `Motor[1].ActPos` , `Motor[1].JogSpeed`)를 들어 답하면 Skill이 동작 중입니다.

6. 권한 승인 이해하기 (처음 사용자 필독)

Claude Code는 파일을 바꾸거나 명령·도구를 실행하기 전에 승인을 묻습니다.

- 프롬프트가 뜨면 **허용(Allow)** 또는 **거부**를 고릅니다. "이 세션에서 항상 허용"을 고르면 같은 종류는 다시 묻지 않습니다.
- MCP 도구(빌드·다운로드·컨트롤러 명령)도 처음 호출할 때 동일하게 승인을 묻습니다 — 안심하고 진행하세요.
- 컨트롤러 상태를 바꾸는 작업(`download_project`, `send_command`)은 **내용을 확인한 뒤** 허용하길 권장합니다.

7. Skill 사용법 — 물어보기만 하면 됩니다

Skill은 따로 호출하지 않습니다. Power PMAC 관련 질문을 하면 Claude가 자동으로 이 지식을 적용합니다.

다루는 주제 (물어보면 정확히 답하는 범위):

주제	예
문법·변수·흐름제어	P/Q/M/L/I 변수, 온라인 vs 버퍼 명령
데이터구조	<code>Structure[index].Element</code> , <code>Sys. / Motor[] / Coord[] / Gate3[]</code> , <code>SAVED/STATUS</code>
모션 프로그램	<code>open prog</code> , linear/circle/pvt/spline, 좌표계·축정의, lookahead, G코드
PLC 프로그램	<code>open plc</code> , 스캔 모델, 타이머, <code>cmd</code> , 시퀀싱
C 프로그래밍	CPLC(실시간) vs capp(백그라운드), C API, CfromScript
함정(gotcha)	태스크 우선순위, save/reset, 단위, 모션 안전, 에러 ID
프로젝트 구조	폴더 구성, <code>.ppproj</code> , <code>pp_proj.ini</code> 로드 순서

예시 프롬프트:

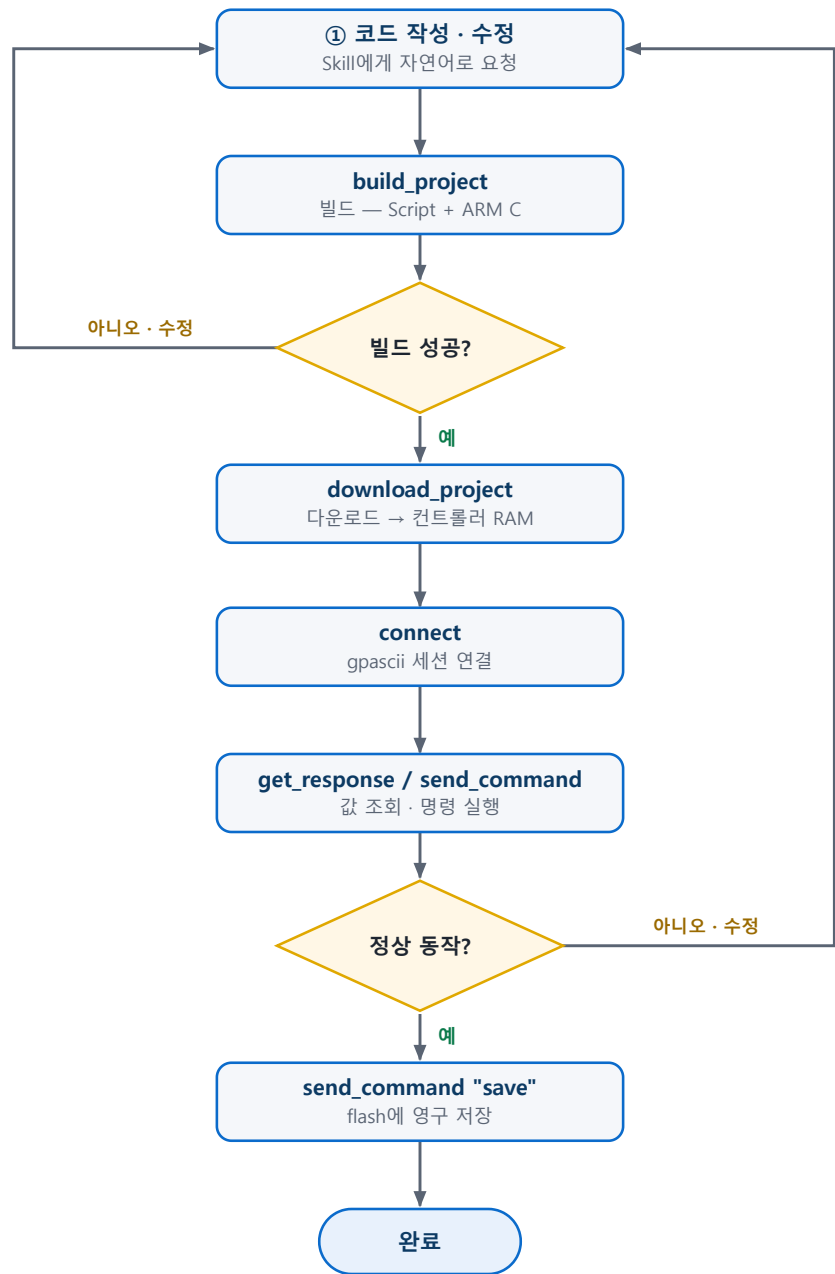
- "1축을 100mm 상대 이동했다가 돌아오는 모션 프로그램 작성해줘"
- "이 PLC가 왜 매 스캔마다 안 도는지 봐줘" (코드 첨부)
- "`Motor[L0+1].ActPos` 처럼 인덱스에 변수 연산을 쓰면 왜 안 돼?" → 인덱스는 상수나 단일 지역변수만 쓸 수 있다는 함정을 설명

- "capp와 CPLC 차이, 언제 무엇을 써야 하는지 알려줘"

동작 확인: 답변이 실제 요소명·firmware 동작과 일치하면 OK. 미심쩍으면 "근거 reference가 어디야?"라고 되물으면 출처를 댁니다.

8. MCP 사용법 — 개발 워크플로

MCP는 도구 이름을 외울 필요가 없습니다. 하고 싶은 일을 자연어로 말하면 Claude가 알맞은 도구를 골라 승인을 받고 실행합니다. 전형적인 개발 흐름은 다음과 같습니다.



전형적인 개발 루프 — 실패하면 코드로 돌아가 다시 빌드, 정상 동작하면 **save** 로 영구 저장.

이렇게 말하면 → Claude가 호출하는 도구:

이렇게 말하면	Claude가 호출
"이 프로젝트 빌드해줘"	<code>build_project</code>
"192.168.0.200에 다운로드해줘"	<code>download_project</code>
"연결하고 Motor[1].ActPos 읽어줘"	<code>connect</code> + <code>get_response</code>
"Motor[1].JogSpeed=10 주고 1축 +방향 조그해줘"	<code>send_command</code> (<code>Motor[1].JogSpeed=10</code> , <code>#1j+</code>)
"Motor[1]~Motor[4] ActPos 한 번에 읽어줘"	<code>get_responses</code>
"Project 폴더 목록 보여줘"	<code>exec_shell</code> (<code>ls -al ...</code>)
"save 해줘"	<code>send_command "save"</code>

도구 전체 목록 (9개):

그룹	도구	하는 일	주요 입력(기본값)
빌드/다운로드	<code>build_project</code>	PC에서 프로젝트를 빌드(컴파일)	<code>projectPath</code> (.ppproj 경로, 필수), <code>configuration</code> (Release)
	<code>download_project</code>	빌드된 프로젝트를 컨트롤러로 전송·로드	<code>projectPath</code> (필수), <code>ipAddress</code> (필수), <code>password</code> (deltatau)
연결	<code>connect</code>	지속 gpascii-셸 세션 열기	<code>ipAddress</code> (필수), <code>username</code> (root), <code>password</code> (deltatau), <code>port</code> (22)
	<code>disconnect</code>	세션 닫기	—
	<code>connection_status</code>	연결 상태 확인	—
명령	<code>send_command</code>	명령 실행(대입·동작; 응답은 상태)	<code>command</code>
	<code>get_response</code>	질의 후 응답 반환	<code>command</code>
	<code>get_responses</code>	여러 질의를 한 번에	<code>commands</code> (배열)
셸	<code>exec_shell</code>	컨트롤러에서 리눅스 셸 명령 실행	<code>command</code> (연결 필요)

기본 접속값: IP는 필수, 사용자 `root`, 비밀번호 `deltatau`, 포트 `22`.

알아둘 점:

- **다운로드는 RAM에만** 올라갑니다(휘발성). 영구 저장은 컨트롤러에서 `save`가 필요합니다 — "save 해줘"라고 시키면 `send_command "save"`로 처리됩니다.
- C 코드를 바꿨으면 `build_project`를 먼저, 그다음 `download_project`. (다운로드는 C를 재빌드하지 않습니다.)
- gpascii 세션은 한 번에 하나입니다. 새로 `connect`하면 이전 세션을 대체합니다.

9. 자주 묻는 질문 · 문제 해결

증상	조치
질문해도 Skill이 안 먹는 듯	Claude Code 재시작 . <code>~/ .claude/skills/powerpmac-dev</code> 연결 확인(없으면 <code>setup.ps1 -SkillOnly</code> 재실행).
<code>claude mcp list</code> 에 <code>powerpmac</code> 이 없음	<code>setup.ps1</code> 재실행 후 재시작 . <code>claude</code> 가 PATH에 없으면 <code>setup.ps1</code> 이 출력한 <code>claude mcp add ...</code> 명령을 직접 실행.
PDK를 못 찾음(빌드 실패)	<code>setup.ps1 -PdkHome "C:\ ... \PDK"</code> (CLLLicFile.lic 있는 폴더 지정). IDE/PDK 설치 확인.
빌드가 파일 잠금 오류	실행 중 MCP가 exe를 잠그므로 Claude Code 종료 후 <code>setup.ps1</code> 재실행.
다운로드 시 컨트롤러 접속 실패	IP/비밀번호(기본 <code>root</code> / <code>deltatau</code> , 포트 22)와 같은 네트워크인지 확인.
도구 실행 승인 프롬프트가 매번 뜸	"이 세션에서 항상 허용"을 선택.

더 깊은 설치·이식성 문제는 루트 `INSTALL.md`, MCP 내부 동작은 `mcp-server/README.md` 를 참고하세요.

10.더 보기

- `README.md` — 저장소 개요
- `INSTALL.md` — 설치·업데이트·문제 해결 상세
- `mcp-server/README.md` — MCP 동작 원리(헤드리스 PTY, rsync + projpp 등)
- `Skills/powerpmac-dev/SKILL.md` — Skill이 참조하는 지식 구조